

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра радіофізики та комп'ютерних технологій

Звіт

З лабораторної №2

З дисципліни «Професійний Python»

**На тему «Поглиблена робота з функціями, структурами даних,
замиканнями, декораторами та функціональними концепціями у
Python»**

Виконав:

Студент 3 курсу

групи ФЕП-31

Войтович М.А.

Перевірив:

проф. Свелеба С. А.

Львів 2026

Варіант: 4 (Система банківських рахунків)

Мета: Ознайомитися з розширеними механізмами роботи з функціями в мові Python, засвоїти передачу довільної кількості аргументів (*args, **kwargs), вивчити принципи створення та застосування замикань (closures) та декораторів, опанувати використання виразів-генераторів, лямбда-функцій, спеціалізованих структур даних (defaultdict, індексовані словники) та синтаксису list comprehensions для побудови ефективного, чистий та модульного коду.

Теоретичні відомості

Аргументи *args та **kwargs:

*args (Positional Arguments): дозволяє передавати у функцію довільну кількість позиційних аргументів у вигляді кортежу (tuple).

**kwargs (Keyword Arguments): дозволяє передавати довільну кількість іменованих аргументів у вигляді словника (dict).

Замикання (Closures):

Конструкція, при якій внутрішня функція запам'ятовує та має доступ до змінних із зовнішньої області видимості (enclosing scope) навіть після того, як зовнішня функція завершила своє виконання.

Декоратори (Decorators):

Функції вищого порядку, які приймають іншу функцію як аргумент, додають нову функціональність (наприклад, логування, вимірювання часу, перевірку прав чи валідацію) та повертають модифіковану функцію без зміни її вихідного коду.

Вирази-генератори (Generator Expressions):

Компактні синтаксичні вирази в круглих дужках (...), які генерують елементи "на вимогу" (lazy evaluation). Вони дозволяють обробляти великі обсяги даних із мінімальним використанням оперативної пам'яті $O(1)$.

defaultdict та швидкі індекси (dict):

defaultdict із модуля collections автоматично створює значення для неіснуючих ключів (наприклад, порожній список list), що спрощує групування даних.

Побудова індексу {account_number: account_dict} дозволяє здійснювати пошук елементів за константний час $O(1)$ замість лінійного перебору $O(n)$.

List Comprehensions та lambda-функції:

List comprehension: лаконічна конструкція для створення нових списків на основі існуючих ітерованих об'єктів із можливістю фільтрації.

lambda: анонімна однорядкова функція, яка найчастіше використовується як кастомний ключ для функцій сортування (sorted(), min(), max()).

Постановка задачі

Модернізувати проєкт **Система банківських рахунків (Варіант 4)**, відмовившись від класів на користь функціонального підходу та використання чистих функцій зі структурами даних на основі списків та словників.

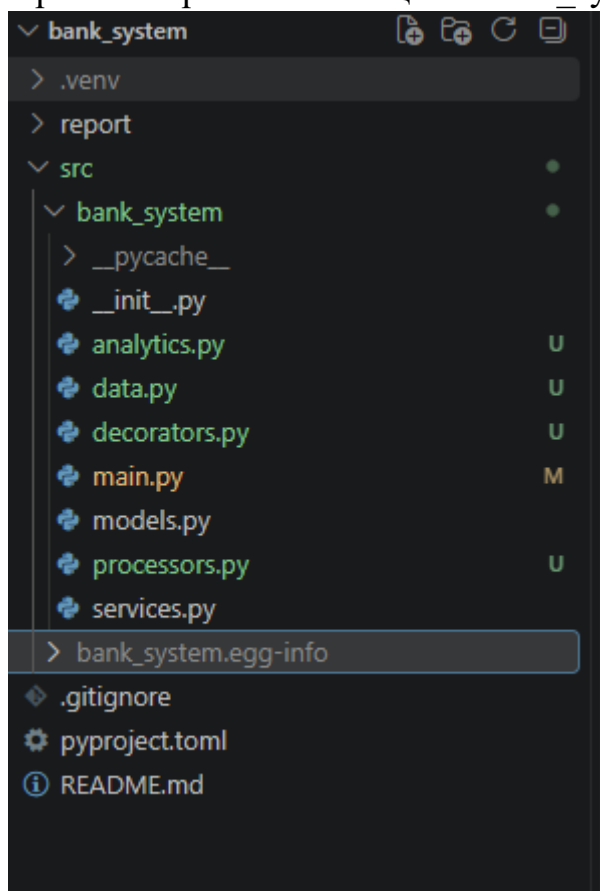
Основні вимоги:

1. Представити кожен рахунок у вигляді словника (dict), а базу рахунків у вигляді списку словників.
2. Реалізувати створення рахунку через функцію з використанням ****kwargs**.
3. Забезпечити ефективний пошук за номером через побудову індексу ($O(1)$) та за ім'ям клієнта через list comprehension.
4. Реалізувати аналітичні функції з обчислення загального капіталу з використанням ***args** та виразів-генераторів.
5. Реалізувати фільтрацію рахунків за допомогою замикання (closure).
6. Реалізувати групування рахунків за типом за допомогою defaultdict.
7. Реалізувати декоратори для логування та валідації вхідних даних.
8. Створити розширене консольне меню (1–9) для демонстрації всіх виконаних вимог.

Хід виконання

1. Структура проєкту (src-layout)

Проєкт збережено в папці `src/bank_system/` та розділено на логічні модулі:



2. Реалізація модулів проєкту

Модуль `src/bank_system/data.py`

Реалізує збереження константних типів рахунків, функцію створення рахунку через ****kwargs** та генерацію початкового списку словників:

```

1 ACCOUNT_TYPES = ("Checking", "Savings", "Credit", "Deposit")
2 def create_account(**kwargs) -> dict:
3     """
4     Створює словник із даними рахунку.
5     Використовує **kwargs для гнучкої передачі параметрів.
6     """
7     required_keys = {"client_name", "account_number", "balance", "account_type"}
8
9     if not required_keys.issubset(kwargs.keys()):
10         raise ValueError(f"Missing required fields: {required_keys - kwargs.keys()}")
11
12     return dict(kwargs)
13
14 def get_sample_accounts() -> list:
15     """Повертає початковий список рахунків (list of dicts)."""
16     return [
17         create_account(client_name="Олександр", account_number="UA1001", balance=15000.50, account_type="Checking"),
18         create_account(client_name="Марія", account_number="UA1002", balance=42000.00, account_type="Savings"),
19         create_account(client_name="Іван", account_number="UA1003", balance=-500.00, account_type="Credit"),
20         create_account(client_name="Олександр", account_number="UA1004", balance=120000.00, account_type="Deposit"),
21         create_account(client_name="Анна", account_number="UA1005", balance=8500.75, account_type="Checking"),

```

Модуль src/bank_system/decorators.py

Містить декоратор логування виклику функцій:

```

1 from functools import wraps
2 import time
3
4 def log_execution(prefix="[LOG]"):
5     """Параметризований декоратор для логування з можливістю вибору префікса."""
6     def decorator(func):
7         @wraps(func)
8         def wrapper(*args, **kwargs):
9             start_time = time.time()
10            print(f"{prefix} Виклик функції: {func.__name__}")
11            result = func(*args, **kwargs)
12            end_time = time.time()
13            print(f"{prefix} {func.__name__} виконано за {end_time - start_time:.6f} сек.")
14            return result
15        return wrapper
16    return decorator

```

Модуль src/bank_system/processors.py

Містить функції побудови швидкого індексу, пошуку через list comprehension, замикання для фільтрації та сортування з використанням lambda:

```

1 from .decorators import log_execution
2
3 @log_execution(prefix="[PROCESSOR]")
4 def build_account_index(accounts: list) -> dict:
5     return {acc["account_number"]: acc for acc in accounts}
6
7 def find_account(index: dict, account_number: str) -> dict:
8     return index.get(account_number)
9
10 def create_min_balance_filter(min_balance: float):
11     """Closure для створення фільтра."""
12     def predicate(account: dict) -> bool:
13         return account["balance"] >= min_balance
14     return predicate
15
16 @log_execution(prefix="[PROCESSOR]")
17 def filter_accounts(accounts: list, predicate_func) -> list:
18     return [acc for acc in accounts if predicate_func(acc)]
19
20 @log_execution(prefix="[PROCESSOR]")
21 def universal_sort(accounts: list, key: str = "balance", reverse: bool = False) -> list:
22     """
23     Універсальне сортування за будь-яким полем словника.
24     """
25     return sorted(accounts, key=lambda acc: acc.get(key, 0), reverse=reverse)

```

Модуль src/bank_system/analytics.py

Реалізує аналітичні обчислення з використанням defaultdict, *args, виразів-генераторів та лямбда-функцій:

```
1  ∨ from collections import defaultdict, Counter, deque
2  ∨ from .decorators import log_execution
3  ∨
4  ∨ operation_history = deque(maxlen=5)
5  ∨
6  ∨ def add_to_history(action: str) -> None:
7  ∨     ∨ """Додає дію до історії (deque)."""
8  ∨     ∨ operation_history.append(action)
9  ∨
10 ∨ def get_history() -> list:
11 ∨     ∨ """Повертає список останніх операцій."""
12 ∨     ∨ return list(operation_history)
13 ∨
14 ∨ @log_execution(prefix="[ANALYTICS]")
15 ∨ def get_account_type_stats(accounts: list) -> Counter:
16 ∨     ∨ """
17 ∨     ∨ Формує статистику кількості рахунків за типами.
18 ∨     ∨ Використовує Counter з модуля collections.
19 ∨     ∨ """
20 ∨     ∨ return Counter(acc["account_type"] for acc in accounts)
21 ∨
22 ∨ @log_execution(prefix="[ANALYTICS]")
23 ∨ def group_accounts_by_type(accounts: list) -> dict:
24 ∨     ∨ grouped = defaultdict(list)
25 ∨     ∨ for acc in accounts:
26 ∨     ∨     ∨ grouped[acc["account_type"]].append(acc)
27 ∨     ∨ return dict(grouped)
28 ∨
29 ∨ def calculate_total_balance(*balances: float) -> float:
30 ∨     ∨ return sum(balances)
31 ∨
32 ∨ def get_average_balance(accounts: list) -> float:
33 ∨     ∨ if not accounts:
34 ∨     ∨     ∨ return 0.0
35 ∨     ∨ return sum(acc["balance"] for acc in accounts) / len(accounts)
36 ∨
37 ∨ def get_max_balance_account(accounts: list) -> dict:
38 ∨     ∨ if not accounts:
39 ∨     ∨     ∨ return {}
40 ∨     ∨ return max(accounts, key=lambda acc: acc["balance"])
```

Модуль src/bank_system/main.py

Реалізує інтерактивне консольне меню користувача (пункти 1–9) для перевірки всіх вимог лабораторної роботи №2:

```

1 import sys
2 from bank_system.data import create_account, get_sample_accounts
3 from bank_system.processors import (
4     build_account_index, find_account,
5     create_min_balance_filter, filter_accounts, universal_sort
6 )
7 from bank_system.analytics import (
8     group_accounts_by_type, calculate_total_balance,
9     get_average_balance, get_max_balance_account,
10    get_account_type_stats, add_to_history, get_history
11 )
12
13 def display_menu() -> None:
14     print("\n=== СИСТЕМА БАНКІВСЬКИХ РАХУНКІВ ===")
15     print("1. Створити новий рахунок (**kwargs)")
16     print("2. Поповнити / Списати кошти")
17     print("3. Знайти рахунок (Dict index)")
18     print("4. Переглянути загальну статистику (*args, Generator)")
19     print("5. Статистика типів рахунків (Counter)")
20     print("6. Фільтрація за мін. балансом (Closure)")
21     print("7. Універсальне сортування (за будь-яким полем)")
22     print("8. Переглянути історію останніх операцій (deque)")
23     print("9. Вийти з програми")
24
25 def main() -> None:
26     accounts = get_sample_accounts()
27
28     while True:
29         display_menu()
30         choice = input("Оберіть дію (1-9): ").strip()
31
32         if choice == "1":
33             acc_num = input("Номер рахунку: ").strip()
34             owner = input("ПІБ власника: ").strip()
35             acc_type = input("Тип (Checking/Savings/Credit/Deposit): ").strip()
36             try:
37                 initial_bal = float(input("Початковий баланс: "))
38                 new_acc = create_account(
39                     account_number=acc_num,
40                     client_name=owner,
41                     balance=initial_bal,
42                     account_type=acc_type
43                 )
44                 accounts.append(new_acc)
45                 add_to_history(f"Створено рахунок {acc_num} ({owner})")
46                 print(f"Створено рахунок {acc_num}!")
47             except ValueError as err:
48                 print(f"Помилка: {err}")
49
50         elif choice == "2":
51             acc_num = input("Введіть номер рахунку: ").strip()
52             index = build_account_index(accounts)
53             acc = find_account(index, acc_num)
54             if not acc:
55                 print("Рахунок не знайдено.")
56                 continue
57
58             action = input("1 - Поповнити, 2 - Списати: ").strip()
59             try:
60                 amount = float(input("Сума: "))
61                 if action == "1":
62                     acc['balance'] += amount
63                     add_to_history(f"Поповнення {acc_num} на {amount} грн")
64                     print(f"Новий баланс: {acc['balance']:.2f} грн")
65                 elif action == "2":
66                     if acc['balance'] < amount:
67                         print("Недостатньо коштів.")
68                     else:
69                         acc['balance'] -= amount
70                         add_to_history(f"Списання {acc_num} на {amount} грн")
71                         print(f"Новий баланс: {acc['balance']:.2f} грн")
72             except ValueError:
73                 print("Некоректне число.")
74
75         elif choice == "3":
76             query = input("Введіть номер або ім'я: ").strip()
77             index = build_account_index(accounts)
78             found = find_account(index, query)
79             if found:
80                 print(f"Знайдено: {found['account_number']} | {found['client_name']} | {found['balance']} грн")
81             else:
82                 print("Нічого не знайдено.")
83
84         elif choice == "4":
85             total = calculate_total_balance(*[a["balance"] for a in accounts])
86             avg = get_average_balance(accounts)
87             max_acc = get_max_balance_account(accounts)
88             print(f"\nЗагальна сума: {total:.2f} грн")
89             print(f"Середній баланс: {avg:.2f} грн")
90             print(f"Найбільший баланс: {max_acc.get('client_name')} ({max_acc.get('balance')} грн)")
91
92         elif choice == "5":
93             stats = get_account_type_stats(accounts)
94             print("\nКількість рахунків за типами (Counter):")
95             for acc_type, count in stats.items():
96                 print(f" - {acc_type}: {count}")

```

```

elif choice == "6":
    try:
        min_val = float(input("Мінімальний баланс: "))
        min_filter = create_min_balance_filter(min_val)
        filtered = filter_accounts(accounts, min_filter)
        print(f"Знайдено {len(filtered)} рахунків:")
        for a in filtered:
            print(f" - {a['client_name']}: {a['balance']} грн")
    except ValueError:
        print("Некоректне число.")

elif choice == "7":
    field = input("Введіть поле для сортування (balance / client_name / account_type): ").strip()
    sorted_accs = universal_sort(accounts, key=field, reverse=True)
    print(f"\nСортування за полем '{field}':")
    for a in sorted_accs:
        print(f" - {a['client_name']} ({a['account_type']}): {a['balance']} грн")

elif choice == "8":
    history = get_history()
    print("\nІсторія останніх операцій (до 5 дій):")
    if not history:
        print("Історія порожня.")
    for i, h in enumerate(history, 1):
        print(f"{i}. {h}")

elif choice == "9":
    print("Вихід з програми.")
    sys.exit(0)

if __name__ == "__main__":
    main()

```

3. Робота з Git

Для відстеження змін у другій лабораторній роботі було виконано серію логічних комітів:

```

(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/data.py src/bank_system/decorators.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано модуль data.py з **kwargs та модуль декораторів decorators.py"
main 64eab4b] Додано модуль data.py з **kwargs та модуль декораторів decorators.py
2 files changed, 38 insertions(+)
create mode 100644 src/bank_system/data.py
create mode 100644 src/bank_system/decorators.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/processors.py src/bank_system/analytics.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано обробку індексів, замикавання, сортування та аналітику з defaultdict і *args"
main a155460] Додано обробку індексів, замикавання, сортування та аналітику з defaultdict і *args
2 files changed, 66 insertions(+)
create mode 100644 src/bank_system/analytics.py
create mode 100644 src/bank_system/processors.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/main.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Оновлено main.py з інтерактивним меню для ЛР2"
main 0e5a2aa] Оновлено main.py з інтерактивним меню для ЛР2
1 file changed, 95 insertions(+), 69 deletions(-)

```

Перевірка історії комітів (git log --oneline):

```

(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git log --oneline
0e5a2aa (HEAD -> main) Оновлено main.py з інтерактивним меню для ЛР2
a155460 Додано обробку індексів, замикавання, сортування та аналітику з defaultdict і *args
64eab4b Додано модуль data.py з **kwargs та модуль декораторів decorators.py
d7bef70 (origin/main) Додано звіт до лабораторної роботи №1
53d3b57 Додано точку входу та документацію застосунку Bank System
603e965 Додано модель банківського рахунку та бізнес-логіку
454af57 Створення структури проекту

```

Індивідуальне завдання (Варіант 4)

Згідно з індивідуальним завданням реалізовано систему управління банківськими рахунками з використанням функціонального підходу Python.

1. **Представлення рахунків:** кожен рахунок зберігається у словнику dict з номером, ім'ям клієнта, балансом і типом рахунку. Усі рахунки об'єднані у список list[dict].
2. **Створення та валідація:** функція create_account(**kwargs) забезпечує гнучке створення рахунків і перевірку обов'язкових полів.

3. **Пошук:** словниковий індекс `build_account_index()` забезпечує пошук за номером рахунку зі складністю ($O(1)$) у середньому. Для пошуку за ім'ям використовується `list comprehension`.
4. **Фільтрація:** замикання `create_min_balance_filter(min_balance)` створює функцію для відбору рахунків за мінімальним балансом.
5. **Аналітика:** функція `calculate_total_balance(*balances)` обчислює загальний баланс, а генератор `(acc["balance"] for acc in accounts)` дозволяє передавати дані без створення проміжного списку.
6. **Групування та сортування:** `defaultdict(list)` групує рахунки за типами, а `lambda` використовується для сортування за балансом.
7. **Логування:** параметризований декоратор `@log_action("Назва дії")` фіксує початок і успішне завершення основних операцій.

Експериментальна частина

Тестування роботи програми та перевірка функціональності консольного меню:

Сценарій 1: Створення рахунку з використанням `**kwargs` (Пункт 1)

```
=== СИСТЕМА БАНКІВСЬКИХ РАХУНКІВ ===
1. Створити новий рахунок (**kwargs)
2. Поповнити / Списати кошти
3. Знайти рахунок (Dict index)
4. Переглянути загальну статистику (*args, Generator)
5. Статистика типів рахунків (Counter)
6. Фільтрація за мін. балансом (Closure)
7. Універсальне сортування (за будь-яким полем)
8. Переглянути історію останніх операцій (deque)
9. Вийти з програми
Оберіть дію (1-9): 1
Номер рахунку: UA1006
ПІБ власника: Maksym Voitovych
Тип (Checking/Savings/Credit/Deposit): Checking
Початковий баланс: 2500
Створено рахунок UA1006!
```

Сценарій 2: Аналітика з використання `*args` та генераторів (Пункт 4)

```
=== СИСТЕМА БАНКІВСЬКИХ РАХУНКІВ ===
1. Створити новий рахунок (**kwargs)
2. Поповнити / Списати кошти
3. Знайти рахунок (Dict index)
4. Переглянути загальну статистику (*args, Generator)
5. Статистика типів рахунків (Counter)
6. Фільтрація за мін. балансом (Closure)
7. Універсальне сортування (за будь-яким полем)
8. Переглянути історію останніх операцій (deque)
9. Вийти з програми
Оберіть дію (1-9): 4

Загальна сума: 187501.25 грн
Середній баланс: 31250.21 грн
Найбільший баланс: Олександр (120000.0 грн)
```

Сценарій 3: Фільтрація за замиканням (Closure) (Пункт 6)

```

=== СИСТЕМА БАНКІВСЬКИХ РАХУНКІВ ===
1. Створити новий рахунок (**kwargs)
2. Поповнити / Списати кошти
3. Знайти рахунок (Dict index)
4. Переглянути загальну статистику (*args, Generator)
5. Статистика типів рахунків (Counter)
6. Фільтрація за мін. балансом (Closure)
7. Універсальне сортування (за будь-яким полем)
8. Переглянути історію останніх операцій (deque)
9. Вийти з програми
Оберіть дію (1-9): 6
Мінімальний баланс: 3
[PROCESSOR] Виклик функції: filter_accounts
[PROCESSOR] filter_accounts виконано за 0.000098 сек.
Знайдено 5 рахунків:
- Олександр: 15000.5 грн
- Марія: 42000.0 грн
- Олександр: 120000.0 грн
- Анна: 8500.75 грн
- Maksym Voitovych: 2500.0 грн

```

Сценарій 4: Групування за типом з defaultdict (Пункт 5)

```

=== СИСТЕМА БАНКІВСЬКИХ РАХУНКІВ ===
1. Створити новий рахунок (**kwargs)
2. Поповнити / Списати кошти
3. Знайти рахунок (Dict index)
4. Переглянути загальну статистику (*args, Generator)
5. Статистика типів рахунків (Counter)
6. Фільтрація за мін. балансом (Closure)
7. Універсальне сортування (за будь-яким полем)
8. Переглянути історію останніх операцій (deque)
9. Вийти з програми
Оберіть дію (1-9): 5
[ANALYTICS] Виклик функції: get_account_type_stats
[ANALYTICS] get_account_type_stats виконано за 0.000173 сек.

Кількість рахунків за типами (Counter):
- Checking: 3
- Savings: 1
- Credit: 1
- Deposit: 1

```

Таблиця вимог до коду

Вимога лабораторної роботи	Стан виконання	Докази / Примітка
Представлення даних через словники (dict)	Виконано	Кожен рахунок є словником, список рахунків — list[dict]
Використання **kwargs	Виконано	Функція create_account(**kwargs) у data.py
Пошук за індексом $O(1)$	Виконано	Функція build_account_index() та find_account() у processors.py

Вимога лабораторної роботи	Стан виконання	Докази / Примітка
Пошук через List Comprehension	Виконано	Пошук за підстрокою імені в main.py
Використання *args	Виконано	calculate_total_balance(*balances) у analytics.py
Вирази-генератори (Generator Expressions)	Виконано	(acc["balance"] for acc in accounts) в main.py та analytics.py
Замикання (Closure)	Виконано	create_min_balance_filter() у processors.py
Використання defaultdict	Виконано	group_accounts_by_type() у analytics.py
Сортування з lambda	Виконано	sort_by_balance() у processors.py та max() в analytics.py
Декоратори	Виконано	Декоратор @log_action у decorators.py

Контрольні запитання

- Що таке list?**
Впорядкована змінювана послідовність елементів.
- Чим list відрізняється від tuple?**
list можна змінювати, а tuple — ні. list використовує [], tuple — ().
- Чому tuple immutable?**
Після створення його елементи не можна змінити.
- Для чого set?**
Для зберігання унікальних елементів і швидкої перевірки наявності.
- Які властивості set?**
Елементи унікальні та мають бути hashable. Порядок не гарантується.
- Що таке dict?**
Структура, яка зберігає пари **ключ–значення**.
- Що таке key-value?**
Кожному унікальному ключу відповідає певне значення.
- Чому пошук у dict швидкий?**
Через використання хеш-таблиці. Середня складність — $O(1)$.
- Що таке hashability?**
Здатність об'єкта мати стабільне хеш-значення.
- Чому list не може бути ключем dict?**
Бо list змінюваний і не є hashable.
- Для чого Counter?**
Для підрахунку кількості повторень елементів.
- Для чого defaultdict?**
Автоматично створює значення за замовчуванням для нового ключа.

13. Для чого deque?

Для швидкого додавання і видалення елементів з обох кінців — $O(1)$.

14. Що таке list comprehension?

Короткий спосіб створення списку, наприклад `[x for x in data]`.

15. Що таке set comprehension?

Короткий спосіб створення множини.

16. Що таке dict comprehension?

Короткий спосіб створення словника.

17. Що таке generator expression?

Вираз, який обчислює значення по одному, не зберігаючи весь результат у пам'яті.

18. Що означає $O(1)$?

Час виконання не залежить від кількості даних.

19. Що означає $O(n)$?

Час зростає пропорційно кількості елементів.

20. Що означає $O(n \log n)$?

Складність, характерна для ефективних алгоритмів сортування.

21. Що означає $O(n^2)$?

Час зростає пропорційно квадрату кількості елементів.

22. Складність пошуку у list?

$O(n)$ — потрібно послідовно перевіряти елементи.

23. Складність пошуку у set?

У середньому $O(1)$.

24. Складність пошуку ключа у dict?

У середньому $O(1)$.

25. Навіщо правильна структура даних?

Вона може значно зменшити час виконання програми.

26. Що таке функціональна декомпозиція?

Розбиття великої задачі на менші окремі функції.

27. Переваги малих функцій?

Код легше читати, тестувати та виправляти.

28. Що таке positional argument?

Аргумент, який передається за позицією.

29. Що таке keyword argument?

Аргумент, який передається за назвою параметра: `name=value`.

30. Для чого *args?

Для передачі довільної кількості позиційних аргументів у вигляді tuple.

31. Для чого **kwargs?

Для передачі довільної кількості іменованих аргументів у вигляді dict.

32. Що таке lambda?

Коротка анонімна функція.

33. Де використовують lambda?

Наприклад, для сортування: `sorted(data, key=lambda x: x["balance"])`.

34. Чому складну логіку не пишуть у lambda?

Вона призначена для простих виразів і стає важкою для читання.

35.Що таке closure?

Вкладена функція, яка запам'ятовує змінні зовнішньої функції.

36.Які змінні пам'ятає closure?

Змінні зовнішньої функції, які використовуються всередині вкладеної.

37.Що таке decorator?

Функція, яка додає іншій функції додаткову поведінку.

38.Для чого декоратори?

Для логування, валідації, авторизації, кешування та вимірювання часу.

39.Для чого functools.wraps?

Щоб зберегти ім'я та інші метадані оригінальної функції після декорування.

40.Навіщо оцінювати складність алгоритму?

Щоб розуміти, наскільки швидко програма працюватиме при збільшенні кількості даних.

Висновок:

У ході лабораторної роботи №2 було поглиблено знання мови Python та функціонального підходу на прикладі проєкту «Система банківських рахунків» (Варіант 4). Було опрацьовано основні та спеціалізовані структури даних: list, tuple, set, dict, defaultdict, Counter і deque. Також досліджено часову складність основних операцій та використання хеш-таблиць для швидкого пошуку.

Опановано comprehensions, генераторні вирази, *args, **kwargs та lambda. Бізнес-логіку було розділено на окремі функції, що зробило програму більш зрозумілою та зручною для тестування.

Також реалізовано замикання для фільтрації рахунків і декоратори для логування дій. У результаті було створено структурований та функціональний програмний комплекс, який відповідає поставленим вимогам лабораторної роботи.